UCS and UTF-8

What Python and Rust each mean by a string, and why a regex engine makes them talk

An explainer for the conversion inside `rurex`, Python bindings to the Rust `regex` crate. Covers Unicode code points, UCS-2 and its failure, the UTF family, CPython's PEP 393 `str`, Rust's `&str`, and why the engine insists on UTF-8.

github.com/lorien/rurex

Contents

1 Characters are not bytes 3
1.1 Code points 3
1.2 Three counts that all call themselves length 3
1.3 An encoding is two decisions 3
2 UCS, and why the name survives 4
2.1 What broke 4
2.2 Where the word reappears: inside CPython 4
3 The UTF family 4
3.1 UTF-8: one to four bytes 5
3.2 UTF-16 and UTF-32 5
3.3 What a matcher actually cares about 5
4 Python: a str is an array of code points 6
4.1 The three kinds 6
4.2 The object, and why its offsets move 6
4.3 Consequences worth knowing 6
4.4 Lone surrogates: Python can hold what cannot be encoded 7
4.5 The UTF-8 cache 7
4.6 What re does with all this 7
5 Rust: a str is UTF-8 7
6 Why the engine wants UTF-8 8
6.1 An automaton reads one symbol at a time 8
6.2 What the crate actually compiles 8
6.3 Five reasons the choice is right 9
6.4 What the choice costs 9
6.5 Why not teach the crate UCS-2 instead 9
7 The bridge: Python str to Rust str 10
7.1 Where the conversion happens 10
7.2 The three ways to get a &str 10
7.3 The second conversion: byte offsets to character offsets 11
7.4 The divergence: strings that cannot be encoded 11
7.5 What the conversion costs in practice 11
8 A worked example, byte by byte 12
9 Two languages, side by side 13
10 Where this shows up in rurex 13
11 Further reading 13
12 Glossary 13
13 If you remember five things 14

1 Characters are not bytes

Say “string” to two programmers and you get two different data structures. Say “character” and you get at least three incompatible answers, all of them defensible. Everything in this document follows from that ambiguity, so it is worth pinning down first.

1.1 Code points

Unicode defines a space of **code points**: integers from U+0000 to U+10FFFF, each carrying a meaning. `U+0041` is LATIN CAPITAL LETTER A. `U+00E9` is LATIN SMALL LETTER E WITH ACUTE. `U+1F642` is SLIGHTLY SMILING FACE. The space has 1,114,112 slots; most are unassigned, and unassigned is not the same as forbidden.

Two thousand and forty-eight of those slots, U+D800 to U+DFFF, are **surrogate** code points. They are not characters, they are never legal in well-formed text, and they exist for exactly one purpose: to let UTF-16 address characters above the 16-bit range. A code point that is not a surrogate is called a **Unicode scalar value**. Rust can only hold scalar values. Python can hold anything.

1.2 Three counts that all call themselves length

A reader looking at é sees one character. The Unicode text may contain one code point (`U+00E9`) or two (`U+0065` followed by `U+0301`, a combining acute accent). Both render identically; they are different strings. A **grapheme cluster** is what a reader perceives, and it may be an arbitrary number of code points: the family emoji with a zero-width-joiner chain is seven.

Below that sits the **code unit**, the thing the computer actually stores. It is one, two, or four bytes wide depending on the encoding, and it is what every memory layout question is really about.

> **THE ONLY RULE THAT MATTERS HERE**
>
> Python counts code points. Rust counts code points when asked, and bytes by default. Neither counts grapheme clusters. A `len()` on the same visible text can legally return one, two, seven, or thirteen.

1.3 An encoding is two decisions

To store code points as units you must fix the unit’s width and the rule that maps code points to sequences of them. That pair of decisions is an encoding, and every property this document later relies on follows from it: how long the string is in memory, whether `s[i]` is constant time, whether a byte offset is meaningful, whether the text can round-trip through a file.

There is no such thing as “the bytes of a string” in the abstract. There are only the bytes of a chosen encoding.

2 UCS, and why the name survives

UCS stands for Universal Character Set, the name ISO/IEC 10646 gives to the same code point space Unicode defines. The suffixes describe storage: UCS-2 is two bytes per code point, UCS-4 is four.

In the early 1990s, sixteen bits looked generous. 65,536 characters, more than any living script needed, and a fixed width means indexing is multiplication: no decoding, no variable stride, no ambiguity. Unicode 1.1 fit inside that assumption.

2.1 What broke

It did not hold. Unicode 2.0, in 1996, extended the space past the Basic Multilingual Plane to U+10FFFF, mostly for historic scripts and emoji-to-come. UCS-2 has no way to express any of it: a fixed-width 16-bit encoding can address exactly 65,536 values and no more. The fixed-width design was not improved, it was abandoned.

The replacement kept the 16-bit unit and dropped the fixed width. UTF-16 stores most code points in one unit and everything above U+FFFF in two, a **surrogate pair**: a high unit from U+D800 to U+DBFF followed by a low unit from U+DC00 to U+DFFF. The formula that packs an astral code point into such a pair is short enough to quote:

```
high = 0xD800 + ((cp - 0x10000) >> 10)
low  = 0xDC00 + ((cp - 0x10000) & 0x3FF)
```

> **UCS-2 AND UTF-16 ARE NOT THE SAME THING**
>
> Both use two-byte units. Only one of them is a complete encoding. The difference is precisely the surrogate mechanism: UCS-2 has no pairs, so it cannot leave the BMP; UTF-16 has pairs, so it can. Naming a storage width is not the same as naming an encoding.

2.2 Where the word reappears: inside CPython

Python 3.3 replaced the old narrow/wide build split (PEP 393) with a per-string fixed width chosen from the string's maximum code point, and named the three cases UCS-1, UCS-2 and UCS-4. That naming is accurate about storage and misleading about intent: CPython's UCS-2 is genuinely UCS-2, with no surrogate pairs, because a string whose maximum code point is below U+10000 can never contain a pair. It is not UTF-16. There is no byte order, no BOM, and nothing is re-encoded when a string is assigned or compared.

3 The UTF family

UTF means Unicode Transformation Format: a rule for turning code points into code units, and back. Three of them matter here.

3.1 UTF-8: one to four bytes

UTF-8 uses bytes, and the number of bytes depends on the code point.

bytes	bit pattern	range	capacity
1	`0xxxxxxx`	`U+0000` – `U+007F`	128
2	`110xxxxx 10xxxxxx`	`U+0080` – `U+07FF`	1,920
3	`1110xxxx 10xxxxxx 10xxxxxx`	`U+0800` – `U+FFFF`	61,440
4	`11110xxx 10xxxxxx 10xxxxxx 10xxxxxx`	`U+10000` – `U+10FFFF`	1,048,576

Figure 1. The four shapes of a UTF-8 sequence. Every continuation byte begins with the bits `10`; no lead byte does.

Read the patterns and one structural fact falls out: a byte that starts with `10` is always a continuation, and a byte that does not is always a lead. That is what **self-synchronizing** means. From any position in a UTF-8 stream you can tell in two bit tests whether you are at a character boundary, and you can never confuse the middle of a sequence with the start of another.

UTF-8 permits exactly one spelling of each code point. Encoders must reject the overlong two-byte spelling of U+0000 (`C0 80`), a continuation byte with no lead before it, a truncated sequence, and any code point above U+10FFFF. Because ASCII occupies bytes below 0x80, and those bytes never appear inside a multi-byte sequence, plain ASCII text is byte-identical in UTF-8 and in the original ASCII.

3.2 UTF-16 and UTF-32

UTF-16 is one 16-bit unit for U+0000 to U+FFFF and a surrogate pair above it. Its unit width is fixed; its **width per code point** is not. UTF-32 is one 32-bit unit per code point, fixed on both counts, four bytes per character whether the text needs it or not. Both of the 16- and 32-bit forms carry a byte order: `FE FF` and `FF FE` at the head of a stream say which one, which is why they are awkward as interchange formats and why neither is used as an in-memory string representation on modern systems.

	UCS-2	UTF-16	UTF-32	UTF-8
unit width	2	2	4	1
units per code point	1	1 or 2	1	1 – 4
reaches U+10FFFF	no	yes	yes	yes
byte order matters	no	yes	yes	no
ASCII is a prefix	no	no	no	yes
self-synchronizing	n/a	no	yes	yes
bytes for ASCII text	2 per char	2 per char	4 per char	1 per char

Figure 2. The four encodings side by side. UCS-2 is fixed and incomplete; UTF-16 is complete but variable per code point.

3.3 What a matcher actually cares about

A pattern matcher reads its input unit by unit, so two of those rows decide how pleasant an encoding is to match in: whether there is a byte alphabet at all, and whether a scanner that lands in the middle of a character can recover.

UTF-8 has a one-byte unit, which gives it an alphabet of exactly 256 symbols, and it is self-synchronizing, which means a scanner can start anywhere and resynchronize with two bit tests. UTF-16 has a 65,536-symbol alphabet and no resynchronization rule: a lone high surrogate looks like an ordinary input symbol until the following unit arrives. UTF-16 also has the property that a code point may span two units, so a matcher must either track pairs or forbid its transitions from ending between them.

4 Python: a str is an array of code points

CPython's `str` is not a byte string. It is a flat array of fixed-width units, each holding exactly one code point, plus a small header. The width is chosen per string, from the largest code point it contains, at the moment it is created.

4.1 The three kinds

- Every code point below U+0080: one byte per unit, and the string is additionally flagged **ASCII**.
- Every code point below U+0100: one byte per unit, no ASCII flag, Latin-1 semantics.
- Every code point below U+10000: two bytes per unit.
- Anything above: four bytes per unit.

The threshold is the maximum, so a string that contains a single CJK character is stored two bytes per unit throughout, even if the rest is ASCII. The choice is sticky: strings are immutable, so it can never become wrong.

text	max code point	kind	CPython bytes	UTF-8 bytes
abc	U+0063	UCS-1, ASCII	3	3
aé	U+00E9	UCS-1	2	3
aβ	U+03B2	UCS-2	4	3
a🙂	U+1F642	UCS-4	8	5

Figure 3. The same short strings, stored the way CPython stores them and the way UTF-8 would. Neither column wins everywhere.

Figure 3 is the whole mismatch in one table. Python's layout makes `len()` a field read and `s[i]` an array index. UTF-8 makes the byte string compact for Latin text and identical to the wire format. The two disagree about how many bytes a character occupies, and neither is wrong.

4.2 The object, and why its offsets move

The header of a compact string holds, in order: the object reference count and type, the length in units, the cached hash, and a word of state bits, followed by the UTF-8 cache fields and then the character data itself.

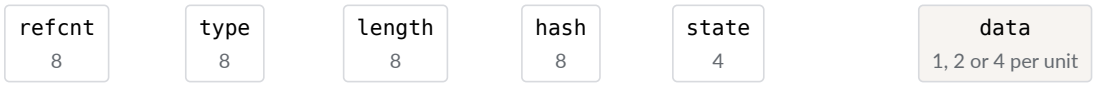


Figure 4. A compact `str`, left to right. The `state` word packs `interned`, `kind`, `compact`, `ascii` and `ready` as bit fields.

Two details of that word matter later. The `kind` field is where 1, 2 or 4 lives, and the `ascii` bit is what lets CPython treat the data as UTF-8 without asking. Both sit at fixed bit positions inside a fixed-size word, and the sizes of everything to the left of them depend on the interpreter: the object header is 16 bytes on a GIL build and 32 on a free-threaded one, and the compact struct carried an extra `wstr_length` field before Python 3.12. Any code that reads these fields through raw pointers is reading a layout that has changed twice within living memory.

4.3 Consequences worth knowing

`len()` is the number of units, which is the number of code points, so it is exact and constant time. Indexing and slicing are array operations with no decoding. Comparison and hashing run over code

points, which is why `"é"` (one code point) and `"é"` (two, decomposed) compare unequal while looking identical.

The cost is memory and interchange. A string of Latin-1 text uses twice the bytes UTF-8 would need if any character in it has a code point above 0xFF, and four times if any character is astral; and nothing outside Python can read the array until it is encoded.

4.4 Lone surrogates: Python can hold what cannot be encoded

Because every unit is a code point and no pair bookkeeping exists, nothing stops a `str` from containing a surrogate. It happens routinely: `"\ud800"` written literally, `surrogatepass` decoding of malformed UTF-8, and surrogateescape decoding of undecodable file names all produce them. Such a string is perfectly valid Python and impossible to encode as UTF-8, which is a rule the encoder enforces rather than a technicality. `s.encode("utf-8")` raises `UnicodeEncodeError` on it.

> **THIS IS THE SHAPE OF THE WHOLE PROBLEM**
>
> A `str` is a sequence of code points that may not be encodable at all, stored in a width that depends on its own contents. Everything downstream that wants bytes has to decode, convert, or refuse.

4.5 The UTF-8 cache

CPython does keep a UTF-8 copy when something asks for one. `PyUnicode_AsUTF8AndSize` encodes the string once, stores the pointer and length in the object's `utf8` and `utf8_length` fields, and returns the same pointer on every later call. Since strings are immutable, the cache never goes stale and is never invalidated.

Two limits are worth stating. It applies to compact strings, which is nearly all of them but not all. And it is filled only on demand: a freshly created non-ASCII string has a null `utf8` pointer, and the first consumer pays for the whole string in one go, at CPython's scalar encoder speed.

4.6 What re does with all this

Nothing. `re` matches a `str` as code points, directly in the storage Python chose, and compiles a `str` pattern into instructions that consume code points. It never builds a UTF-8 copy, and it never needs one: its engine was written for code points, and for `bytes` patterns it has a separate byte mode.

That is the asymmetry at the heart of this document. Python's own regex engine and Python's string storage were designed for each other. A third-party engine that matches bytes has to cross the boundary, and crossing it means encoding.

5 Rust: a str is UTF-8

Rust went the other way. `str` is a byte slice with an invariant attached: the bytes are always valid UTF-8. Every safe constructor enforces it, so any function that receives `&str` may assume it without checking.

```
let s = "aé";
s.len();              // 3 — bytes, not characters
s.chars().count();    // 2 — code points, an O(n) walk
&s[0..1];             // "a", a char boundary
&s[0..2];             // panics: byte 2 is inside "é"
```

`char` is a Unicode scalar value stored as four bytes. It cannot be constructed from a surrogate. Indexing a `str` is by byte offset and panics if the offset is not a boundary; iterating is by `chars()`, which decodes as it goes.

The practical effect is that Rust's in-memory string **is** the interchange format. Handing a `&str` to a byte-oriented matcher costs nothing, because no conversion exists to perform. The corresponding cost lands on anyone who wants a character index: `chars().count()` is a linear scan, exactly as `encode("utf-8")` is in Python.

> **AN ASYMMETRY WITH NO WORKAROUND**
>
> Rust's `str` cannot represent a lone surrogate. Python's `str` can. A conversion from Python to Rust is therefore partial: some Python strings have no `&str` at all, and no amount of cleverness produces one. The only honest outcomes are an error or a lossy substitution.

6 Why the engine wants UTF-8

The previous two sections described two representations that are individually sensible. This one explains why the Rust engine cannot simply be pointed at the Python one, and why the demand for UTF-8 is a design choice rather than a limitation of the hardware.

6.1 An automaton reads one symbol at a time

A regex engine is a state machine. A pattern compiles into states and transitions, and the transitions are labelled with the symbols the machine expects to read. The set of those symbols is the **alphabet**, and the alphabet is the single most consequential decision in the implementation.

A literal `hello` is five transitions. The class `[a-z]` is twenty-six. The wildcard `.` is one transition labelled "every symbol". Nothing in that description mentions characters, bytes, or Unicode: the machine operates on whatever alphabet it was given.

Give it bytes and the alphabet has 256 symbols. Dense transition tables are feasible, every symbol fits in a byte, and a state's table is small enough to sit in cache. Give it 16-bit units and the alphabet has 65,536 symbols: tables become sparse range maps or hash lookups, and — worse — a single astral character occupies two consecutive symbols, so every transition into or out of a class has to know whether it may stop between a high and a low surrogate.

6.2 What the crate actually compiles

The Rust `regex` crate parses a pattern into an intermediate representation in which character classes are still sets of code points, then expands those sets into **byte** ranges, and only then builds the automaton. A range like U+0080 to U+07FF becomes the set of two-byte sequences that encode it; the expansion code began life in a crate called `utf8-ranges`.

The consequence is worth stating plainly: a pattern given as `&str` compiles to an automaton over bytes that can only accept complete, well-formed UTF-8 sequences. It cannot match a byte inside a character, because no accepting state is reachable there. The engine is a UTF-8 automaton, not a byte matcher with a validation pass bolted on.

```
h.llo      h    then 1-4 UTF-8 bytes then    llo
é          C3 A9
\p{Greek}  a set of 2-byte sequences in CE..CF and D0..D1
[0-9]      the single byte range 30..39
```

6.3 Five reasons the choice is right

1. **One alphabet covers everything.** With bytes, there is exactly one automaton shape for all of Unicode. With 16-bit units, every character class has to be compiled twice: once for the BMP portion, once for the astral portion expressed as surrogate pairs.
2. **ASCII stays cheap.** Patterns for ASCII text compile to plain byte sequences, and the engine's prefilters can scan for them with `memchr` and SIMD, skipping sixteen to sixty-four bytes per instruction. In UTF-16 the same text occupies two bytes per character, so every skip loop moves half as many characters per byte read.
3. **Self-synchronization makes boundaries cheap.** Two bit tests decide whether a byte offset is a character boundary. Empty matches, word boundaries, and the rule that a match may not split a character are all expressible on top of that test. In UTF-16 a lone high surrogate is indistinguishable from a normal symbol until the next unit arrives.
4. **No byte order.** UTF-8 has no endianness, so a buffer written on one machine is readable on another and the automaton needs no swap mode.
5. **It is the native form.** Rust's `&str` is UTF-8 already, so the engine receives exactly what the caller has, with no conversion anywhere in Rust.

There is a sixth, quieter benefit: the same engine, in its `bytes` variant, can match arbitrary non-UTF-8 input without a second implementation. One automaton compiler serves both the text and the byte API.

6.4 What the choice costs

Honesty requires the other column. A variable-length encoding makes character classes expand: `[α-ω]` is not twenty-five transitions but a set of byte-range automata sharing prefixes, and that is more states than a code-point automaton would need. Positions reported by the engine are byte offsets, so any API that speaks in character indices has to pay for a mapping. And constructs that want to look backwards, or to know the length of a character before matching it, are harder than in a fixed-width encoding.

For the workloads in question — mostly ASCII, with runs of Cyrillic, Arabic, CJK, or emoji — the first three reasons dominate and the price is small. For an engine embedded in a language whose strings are already UTF-16, such as JavaScript, Java, or .NET, the balance comes out differently, which is why those engines match 16-bit units instead.

6.5 Why not teach the crate UCS-2 instead

It is not that a UCS automaton is impossible. It is that it would be a second engine. The crate exposes exactly two alphabets: valid UTF-8 through `Regex`, and raw bytes through `bytes::Regex`. There is no UTF-16 or UCS input mode to switch on.

Adding one would mean: compiling every character class into surrogate-pair-aware transitions, doubling the class expansion work, adding endianness handling on the haystack side, and maintaining two code paths through the automaton compiler for the rest of the crate's life. Even then, the UCS path would be slower on the payloads that matter, because two-byte units halve the number of characters a SIMD loop can skip per byte read.

rurex's entire premise is reuse: a small binding over a mature engine, with Python's API on top. Rewriting the engine to consume UCS would delete the reason the project exists. The conversion is the price of the reuse, and the rest of this document is about making that price small.

7 The bridge: Python str to Rust str

7.1 Where the conversion happens

A pattern crosses the boundary once, when it is compiled, and the cost is irrelevant: patterns are short and the result is cached. A haystack crosses on every call.

step	what happens	cost
1	The Python `str` arrives as a PEP 393 object: a kind and an array of units.	zero
2	The storage is inspected: is it ASCII, is the UTF-8 cache warm, is the layout validated?	a few loads
3	A `&str` is produced by borrowing, or by transcoding into a scratch buffer.	see below
4	The engine matches over UTF-8 bytes and reports byte spans.	the search
5	Byte spans are converted to code point offsets for Python.	one walk

Figure 5. The path a single call takes. Steps 1 and 5 are the two conversions this document is about; step 4 is the matching the caller actually wanted.

7.2 The three ways to get a &str

path	condition	cost
Borrow UCS-1	the string is ASCII, so its units are already UTF-8	zero
Borrow cache	non-ASCII, and some earlier consumer filled CPython's UTF-8 cache	zero
Transcode	non-ASCII with a cold cache: units are widened and re-encoded into a reuseable scratch buffer	1.4 – 3.8 GB/s
Ask CPython	the object layout could not be validated, so `PyUnicode_AsUTF8AndSize` encodes and caches	0.45 GB/s

Figure 6. The four possible sources of a `&str`, in the order they are tried. The last one is the fallback that always works and is always correct.

The first path is not a trick: CPython's own ASCII fast paths rely on the same fact. The second exists because UTF-8 copies are commonly already present, produced by JSON decoding, database drivers, or an earlier rurex call. The third is the case the optimisation exists for, and the last is what keeps a future CPython layout change from turning into a wrong answer.

> **THE CACHE IS READ-ONLY TERRITORY**
>
> CPython's `utf8` buffer is allocated with `PyMem_Malloc` and belongs to the object. Only `PyUnicode_AsUTF8AndSize` may fill it, under that allocator's own rules. A binding may read the pointer and must not write through it, which also means rurex cannot leave a UTF-8 copy behind for the next call. Strings warmed only by rurex are re-encoded on the next scan.

7.3 The second conversion: byte offsets to character offsets

The engine answers in bytes. Python asks in code points. For a haystack `aβc` and a match on `β`, the engine reports the byte span `(1, 3)` while Python expects `(1, 2)`.

Converting per span would rescan the prefix each time, which is quadratic for a `findall` over a long string. rurex instead collects every offset of the result, sorts and deduplicates them, and walks the string once from offset to offset, counting characters. The total work is one pass over the text up to the last reported offset, no matter how many groups or matches there are.

The same conversion runs in reverse for `pos` and `endpos`, which Python defines as character indices: rurex walks `char_indices` up to the farther of the two and uses the byte offsets it finds. Both directions are linear and both are skipped entirely when the count is trivial, which is why an ASCII haystack costs nothing extra.

7.4 The divergence: strings that cannot be encoded

One class of Python strings has no UTF-8 form at all: those containing a lone surrogate. CPython's encoder refuses them, and so must any correct binding: the transcode path raises `UnicodeEncodeError`, with the same message a direct `encode("utf-8")` would produce.

This is a genuine behavioural difference from `re`, which matches such strings happily because it never encodes anything. It is also unavoidable short of writing a UCS engine, so it is documented rather than worked around. Strings without lone surrogates — effectively all of them — are unaffected.

7.5 What the conversion costs in practice

operation	throughput	context
CPython encoder, cold string	0.45 GB/s	in-call, GIL held
Rust scalar encoder	0.48 GB/s	any architecture
AVX2 transcode, mixed Arabic	1.9 GB/s	x86_64, AVX2 + BMI2
AVX2 transcode, pure CJK	3.8 GB/s	x86_64, AVX2 + BMI2
AVX2 transcode, emoji	1.5 GB/s	x86_64, AVX2 + BMI2

Figure 7. Measured on 1M short Telegram posts. The kernel buys 3 to 8 times the scalar encoder, which is what turns a x3.0 regression into x1.6.

The end-to-end effect on that corpus: 1.46 s before the transcoder existed, 0.77 s after, against `re`'s 0.48 s, with the GIL held and one thread. The remaining gap is the encoding that still has to happen, plus the offset mapping; there is no arrangement of the current engine that removes it.

8 A worked example, byte by byte

Everything above compresses into one trace. Take `s = "aβ7c"` and `p = rurex.compile(r"β\d")`, then call `p.search(s)`.

stage	a	β	7
code points	U+0061	U+03B2	U+0037
UCS-2 units	00 61	03 B2	00 37
UTF-8 bytes	61	CE B2	37

Figure 8. The haystack at each stage. CPython stores four UCS-2 units; the engine sees five UTF-8 bytes; the answer comes back in code point indices.

1. **Python hands over a str.** Its maximum code point is U+03B2, so the kind is UCS-2: four units, eight bytes of storage, no ASCII flag. Its UTF-8 cache is null because nothing has encoded it yet.
2. **rurex reads the layout.** Not ASCII, cache cold: path 3. The prefilter, if the pattern has one, would scan these units first; this pattern needs no such shortcut, so the string is transcoded into a scratch buffer.
3. **The engine matches.** It sees `61 CE B2 37 63` and reports the match as the byte span `(1, 4)`, because β7 occupies bytes 1 through 3 inclusive.
4. **rurex maps the span back.** The distinct offsets are 1 and 4. Walking the UTF-8 text once: one character precedes byte 1, and three precede byte 4, so the span becomes `(1, 3)` — which is exactly what Python's `re` would report.
5. **A bounded call does the same work twice.** `p.search(s, 2)` means "start at code point 2", which is byte offset 3. `pos` is converted forward, the search runs in bytes, and any span is converted back.

> **THE INVARIANT THE WHOLE DESIGN PROTECTS**
>
> Every conversion is one-directional and total. Nothing outside `utf8` reads UCS, nothing inside the engine reads it either, and the mapping back is a single walk whose result is checked against the same rules CPython uses. A wrong answer would require the layout detection, the transcoder and the offset walk to fail in agreement.

9 Two languages, side by side

	Python `str`	Rust `&str`
storage	UCS-1, UCS-2 or UCS-4, fixed per string	UTF-8 bytes
unit	one code point	one byte
`len()` means	code points	bytes
indexing	constant time	only at character boundaries
lone surrogate	allowed	impossible
ASCII text	1 byte per character	1 byte per character
non-Latin text	1, 2 or 4 bytes per character	1 – 4 bytes
regex engine input	code points	bytes
what a file needs	an encode call	nothing

Figure 9. The same string, described by the two runtime models that have to meet inside rurex.

10 Where this shows up in rurex

Everything in this document lives in the rurex repository:

- `src/utf8.rs` — the three ways to obtain a `&str`, and the transcoder behind the cold path.
- `src/utf8/simd.rs` — the AVX2 kernel and the scalar reference it is tested against.
- `src/regex.rs` — the span walk and the `pos` / `endpos` conversion.
- ADR-0012 and ADR-0013 — the decisions, and the alternatives they rejected.

11 Further reading

- The Unicode Standard, chapter 2, **General Structure** — code points, units, and the encodings.
- RFC 3629 — UTF-8, its four shapes, and the exact list of forbidden forms.
- PEP 393 — **Flexible String Representation**, the rationale for CPython's three kinds.
- The `regex` crate's documentation on Unicode support and on `regex::bytes`.
- **UTF-8 Everywhere** — the argument for UTF-8 as the in-memory format, with the opposite trade-offs spelled out.

12 Glossary

Code point an integer in the Unicode space, U+0000 to U+10FFFF, carrying a meaning. Not the same as a visible character.

Scalar value a code point that is not a surrogate, and therefore encodable. Rust's `char` holds exactly these.

Code unit the storage element of an encoding: a byte in UTF-8, a 16-bit word in UTF-16 and UCS-2, a 32-bit word in UTF-32.

BMP the Basic Multilingual Plane, U+0000 to U+FFFF — everything a 16-bit fixed-width encoding can reach.

Surrogate a code point in U+D800 to U+DFFF, unusable alone, existing so that a **pair** of them can encode beyond the BMP in UTF-16.

UCS-2 a fixed-width, 16-bit encoding of the BMP only. Cannot express a surrogate pair, therefore cannot express astral characters.

UTF-16 a 16-bit-unit encoding of the whole space, using one unit or a surrogate pair. Variable width per code point.

UTF-8 a byte encoding of the whole space in one to four bytes, self-synchronizing, ASCII-compatible, endianness-free.

PEP 393 the Python 3.3 change to store `str` as PEP 393 UCS with a per-string kind, replacing the narrow/wide build split.

Grapheme cluster what a reader perceives as one character. May be any number of code points.

Self-synchronizing the property that a byte offset can be classified as a character boundary from a couple of bits, without context.

Overlong form a longer-than-necessary encoding of a code point. Forbidden in UTF-8; decoders must reject it.

13 If you remember five things

---

1. **A code point is a number; an encoding is a storage rule.** UTF-8 uses one to four bytes, UTF-16 one or two 16-bit units, UCS-2 exactly one 16-bit unit and nothing more.
2. **UCS-2 died of arithmetic,** not of neglect: 16 fixed bits cannot reach above U+FFFF. UTF-16 survived by adding surrogate pairs, which is exactly what turned it from fixed to variable width.
3. **Python's `str` is an array of code points,** in the narrowest fixed width that holds the string's largest code point. `len()` counts code points, indexing is constant time, and the result is not bytes — it may not even be encodable.
4. **Rust's `str` is UTF-8, always,** and its `char` is a scalar value. Strings that Python accepts and Rust cannot represent do exist, and the boundary has to say so.
5. **The engine compiles to byte automata** because 256 symbols are cheap, ASCII keeps its fast paths, and self-synchronization makes boundaries decidable. So rurex converts the haystack on the way in and the offsets on the way out. That is the whole price of reuse.